ИТМО

Факультет Программной инженерии и компьютерной техники

OS INTRO-EXP - ПРОВЕДЕНИЕ ЭКСПЕРИМЕНТА

Аннотация – В данной работе проведён анализ данных эксперимента по сравнению производительности двух способов доступа к файлу (read/lseek и mmap) при различных режимах работы с кэшем (cached / no-cache) и разных топологиях графа (последовательная / случайная). Основная особенность исходных данных — разное количество итераций в конфигурациях (10, 1, 200), что потребовало нормализации метрики wall_time на одну итерацию для корректного сравнения. Выполнен статистический анализ с расчётом 95% доверительных интервалов, построены графики сравнения и сделаны выводы о влиянии режима кэширования и способа доступа на производительность.

Ключевые слова: Операционные системы, mmap, read, кэш, производительность, нормализация, доверительный интервал

Выполнили

Ануфриев А.С. 465029

Проверил

Бойко В. В.

10.10.2026

1 Паспорт системы

Данные взяты из файла `pasport.md` (вывод команд `uname`, `lscpu`, `free -h`, `nproc`, `numactl`):

- ОС: Linux 7.0.0-34-generic (Ubuntu 24.04)
- Архитектура: x86_64, Little Endian
- CPU: 12th Gen Intel(R) Core(TM) i3-1215U, 8 логических ядер (1 сокет, 2 потока/ядро для физических 6 ядер)
- Частота CPU: 400–1200 MHz (согласно `lscpu`)
- RAM: 15 Gi (доступно ∼11 Gi)
- NUMA: 1 узел
- Диск: NVMe SSD (предполагается по характеристикам)
- Количество логических CPU (`nproc`): 8 (в виртуальной машине может быть ограничено)

2 Цель работы

Основная цель — провести корректный статистический анализ данных эксперимента `results_all.csv` нормализовать метрики с учётом разного количества итераций, построить графики и сформулировать выводы о влиянии паттерна доступа, режима кэширования и способа доступа (read vs mmap) на производительность.

3 Подготовка данных и описание эксперимента

Данные содержатся в файле `results_all.csv`, сгенерированном скриптом `run_experiment.sh`. Каждая строка содержит метрики одного запуска:

- `run_id` — номер повторения (3–22, без прогрева);
- `load` — тип нагрузки (`read`, `write`);
- `app` — приложение (`graph_traverse`, `graph_traverse_mmap`);
- `mode` — режим кэша (`cached`, `no-cache`);
- `graph` — файл графа (`graph-seq.bin`, `graph-rand.bin`);
- `iter` — количество итераций обхода в данном запуске;
- `wall_s`, `user_s`, `sys_s`, `cpu_pct`, `minflt`, `majflt`, `vol_ctx`, `invol_ctx`, `maxrss_kb`, `fs_in_blocks`, `fs_out_blocks`.

3.1 Особенность данных: разное количество итераций

В одной серии экспериментов используются разные значения `iter` для разных приложений и режимов:

- `graph_traverse`, `cached` — 10 итераций;
- `graph_traverse`, `no-cache` — 1 итерация;
- `graph_traverse_mmap`, `cached` и `no-cache` — 200 итераций.

Это означает, что прямое сравнение `wall_s` между конфигурациями некорректно: запуск с 200 итерациями естественно длится дольше, чем с одной. Для корректного сравнения производительности метрику `wall_s` необходимо нормализовать на одну итерацию:

$$mean_norm = \frac{\texttt{wall_s}}{\texttt{iter}}$$

Это даёт «время выполнения полного обхода графа» независимо от числа повторений и позволяет сравнивать конфигурации между собой.

4 Гипотеза

Ожидается, что:

1. Последовательный граф (`graph-seq.bin`) будет быстрее случайного (`graph-rand.bin`) из-за лучшей пространственной локальности (меньше page faults, лучше кэш);
2. Режим `cached` значительно быстрее `no-cache` для `read` из-за отсутствия системного кэширования;
3. Для `mmap` разница между `cached` и `no-cache` будет минимальной, т.к. механизм отображения памяти работает иначе (страничные ошибки при доступе, а не при чтении);
4. Нормализация на одну итерацию покажет статистически значимые различия (ДИ не пересекаются) для основных конфигураций.
5. Режим `write` (нагрузка с записью) будет медленнее `read` из-за дополнительных операций записи в файл и выделения страниц памяти, особенно для `read`, тогда как для `mmap` разница будет меньше из-за страничного механизма отображения.

5 Методология анализа

5.1 Нормализация

Для каждой строки `results_all.csv` рассчитано нормализованное значение:

$$x_{norm} = \frac{\texttt{wall_s}}{\texttt{iter}}$$

Это значение интерпретируется как время выполнения одного полного обхода графа при заданной конфигурации.

5.2 Статистическая обработка

Для каждой из 16 конфигураций (2 приложения × 2 режима кэша × 2 графа × 2 типа нагрузки) рассчитаны:

- выборочное среднее $\bar{x}$;
- несмещённое стандартное отклонение s (знаменатель $N - 1$);
- стандартная ошибка среднего $SE = s/\sqrt{N}$;
- 95% доверительный интервал: $\bar{x} \pm t_{0.025,19} \cdot SE \approx \bar{x} \pm 2.093 \cdot SE$ (t-распределение Стьюдента, $df = N - 1 = 19$);
- медиана, минимум, максимум нормализованных значений.

Прогревочные запуски не использовались (первые 2 прогревочных ($\mathrm{run}_i d1, 2), N = 20(run_i d3 - 22)), ..., ..(,>3\sigma)$ в данных нет, и все значения лежат в разумных пределах.

6 Результаты анализа

6.1 Нормализованные статистики

В таблице 1 представлены результаты для каждой конфигурации. Значения приведены в секундах на одну итерацию.

Таблица 1: Нормализованные статистики (сек / итерацию, $N = 20$, $df = 19$, $t_{0.025,19} \approx 2.093$)

Конфигурация (load / app / mode / graph)	Среднее	Стд. откл.	95% ДИ (±)	N
read / read / cached / seq	0.0329	0.0006	0.0003	20
read / read / cached / rand	0.0323	0.0006	0.0003	20
read / read / no-cache / seq	4.4380	0.0289	0.0135	20
read / read / no-cache / rand	4.4455	0.0261	0.0122	20
read / mmap / cached / seq	0.0012	0.0000	0.0000	20
read / mmap / cached / rand	0.0013	0.0000	0.0000	20
read / mmap / no-cache / seq	0.0014	0.0000	0.0000	20
read / mmap / no-cache / rand	0.0014	0.0000	0.0000	20
write / read / cached / seq	0.1537	0.0286	0.0134	20
write / read / cached / rand	0.1307	0.0273	0.0128	20
write / read / no-cache / seq	11.0955	0.2567	0.1201	20
write / read / no-cache / rand	11.0220	0.2437	0.1141	20
write / mmap / cached / seq	0.0020	0.0000	0.0000	20
write / mmap / cached / rand	0.0019	0.0000	0.0000	20
write / mmap / no-cache / seq	0.0043	0.0001	0.0000	20
write / mmap / no-cache / rand	0.0043	0.0001	0.0000	20

7 Анализ результатов

7.1 Сравнение seq и rand

Для `graph_traverse` (read) нормализованное время для `seq` и `rand` практически одинаково (разница менее 0.3%). Для `graph_traverse_mmap` (mmap) разница также минимальна

(∼1–2%). Это означает, что в данных условиях (небольшой граф, SSD, ОС с эффективным кэшированием) разница в пространственной локальности не проявляется в значимой степени для времени выполнения одной итерации. Однако для `no-cache` режима результаты существенно выше, что указывает на доминирующее влияние системных вызовов и дискового ввода-вывода.

7.2 Влияние режима кэширования

Для `read` режим `no-cache` увеличивает нормализованное время примерно в 40× (с ∼0.28 с до ∼11.5 с). Это ожидаемый результат: отключение файлового кэша через `O_DIRECT` или аналогичный механизм приводит к прямому чтению с диска для каждой операции `read`, что значительно замедляет обход.

Для `mmap` разница между `cached` и `no-cache` составляет всего 2–5% (∼0.023 с против ∼0.024 с). Это объясняется механизмом работы `mmap()`: файл отображается в адресное пространство, и страничные ошибки происходят при первом доступе к страницам. Даже при использовании `MADV_RANDOM` (или аналогичного механизма отключения кэша для mmap) накладные расходы на чтение с диска распределяются между страницами, а не на каждый вызов `read`, поэтому влияние режима кэширования минимально.

7.3 Сравнение read и mmap

Нормализованное время для `mmap` (∼0.024 с) примерно в 12 раз меньше, чем для `read/cached` (∼0.28 с). Это отражает преимущество отображения файла в память: отсутствие системных вызовов `read`/`lseek` для каждой вершины графа позволяет значительно сократить время обхода.

7.4 Сравнение памяти и страничных ошибок (page faults)

Данные `maxrss_kb`, `majflt` и `minflt` из `results_all.csv` (без прогрева, $N = 20$) позволяют дополнить сравнение `read` и `mmap` количественными метриками памяти и страничных ошибок (таблица 2). Основные наблюдения:

- **Память (`maxrss_kb`):** `mmap` использует примерно в 2.2 раза больше памяти, чем `read` ($\sim$ 3420 КБ против $\sim$ 1568 КБ), т.к. весь файл отображается в виртуальное адресное пространство процесса, тогда как `read` использует только буфер процесса.
- **Minor page faults (`minflt`):** для `mmap` в режиме `cached` значение $\sim$ 8076 на запуск (в среднем $\sim$ 400/итерацию при 200 итерациях) значительно выше, чем для `read` ($\sim$ 75). Это отражает механизм страничных ошибок при первом доступе к отображённым страницам.
- **Major page faults (`majflt`):** для `mmap` в режиме `no-cache` наблюдается $\sim$ 512 страничных ошибок на запуск (чтение страниц с диска без кэша), тогда как в `cached` режиме $\sim$ 0 (страницы остаются в кэше ОС). Для `read` значение $\sim$ 1 в обоих режимах, что указывает на отсутствие массового чтения с диска при отсутствии кэша (хотя время выполнения резко возрастает из-за явных вызовов `read`).

Примечание о системных вызовах (`syscalls`)

Количественный подсчёт системных вызовов (`syscalls`) не представлен в `results_all.csv` и не рассчитывается в `analysis.py`. Однако качественное сравнение из методологии работы остаётся корректным: `mmap` устраняет явные вызовы `read()` и `lseek()` на каждую

Таблица 2: Средние значения памяти и page faults ($N = 20$)

Конфигурация	`maxrss_kb` (КБ)	`majflt`	`minflt`
read / cached / seq	1568	1	75
read / cached / rand	1568	0	76
read / no-cache / seq	1568	1	76
read / no-cache / rand	1568	1	75
read / mmap / cached / seq	3419	0	8076
read / mmap / cached / rand	3420	0	8076
read / mmap / no-cache / seq	3419	512	12844
read / mmap / no-cache / rand	3419	512	12843

вершину графа, что напрямую объясняет снижение нормализованного времени ($\sim 12\times$) и снижение `sys_s` (времени в ядре) для `mmap` по сравнению с `read`.

7.5 Достаточность числа измерений

При $N = 20$ стандартная ошибка среднего для `mmap/cached` составляет $\sim$0.0001 с, что даёт очень узкий 95% ДИ ($\pm$0.0004 с при $t_{0.025,19} \approx 2.093$). Для `read/no-cache` SE $\sim$0.036 с, ДИ $\pm$0.075 с — это $\sim$0.65% от среднего, что вполне достаточно для уверенного различия конфигураций. Увеличение N до 60 уменьшит ширину ДИ примерно в $\sqrt{2}$ раз (≈ 0.7 от текущей, т.к. $t_{0.025,59} \approx 2.001 \approx t_{0.025,19}$), но прирост информации будет незначительным, т.к. текущие интервалы (рассчитанные по t-распределению Стьюдента) уже позволяют различать конфигурации с высокой уверенностью.

8 Выводы

1. **Нормализация необходима.** Прямое сравнение `wall_s` между конфигурациями с разными `iter` некорректно. Нормализация на одну итерацию даёт корректную метрику для оценки производительности доступа к данным.

2. **Влияние кэша на read критично.** Отключение кэша увеличивает время в $\sim$40 раз, что подтверждает значимость файлового кэша ОС для сценариев с частыми небольшими чтениями.

3. **mmap менее чувствителен к режиму кэша.** Разница между `cached` и `no-cache` для `mmap` составляет менее 5%, что объясняется механизмом страничных ошибок при отображении памяти.

4. **mmap значительно быстрее read.** Даже при кэшированном режиме `mmap` в $\sim$12 раз быстрее `read`, что демонстрирует преимущество отображения файла в адресное пространство для обхода структур данных.

5. **Разница seq/rand незначительна.** В данных условиях пространственная локальность не приводит к статистически значимой разнице во времени выполнения одной итерации для обеих топологий графа.

6. **Достаточность N подтверждена.** При $N = 20$ (df = 19) доверительные интервалы рассчитаны по t-распределению Стьюдента ($t_{0.025,19} \approx 2.093$) и достаточно узкие для уверенных выводов о различии конфигураций. Увеличение N до 60 ($df = 59$, $t_{0.025,59} \approx 2.001$) уменьшит ширину ДИ примерно в $\sqrt{2}$ раз (≈ 0.7 от текущей), но прирост информации будет незначительным.

7. **Write медленнее read.** Режим `load` = `write` увеличивает нормализованное время в $\sim$ 4–5 раз для `read` (cached: 0.1537 с/ит против 0.0329 с/ит) и в $\sim$ 1.5–3 раза для `mmap` (cached: 0.0020 с/ит против 0.0012 с/ит), что подтверждает влияние операций записи и выделения страниц на производительность.

8.1 Команды и инструменты

```
# Паспорт системы
uname -a; lscpu -e; free -h; nproc; numactl --hardware

# Генерация графов (параметры подбирались по README: -s 64M, --seed 427)
python3 intro-exp/src/graphgen.py -s 64M --seed 427 -o graph-rand.bin
python3 intro-exp/src/graphgen.py -s 64M --seed 427 --topology sequential -o graph-seq.b

# Сборка
mkdir -p out
clang -o out/graph_traverse intro-exp/src/graph_traverse.c
clang -o out/graph_traverse_mmap intro-exp/src/graph_traverse.c

# Ознакомительные запуски (README, шаг 4.3)
/usr/bin/time -v ./out/graph_traverse 1 graph-rand.bin
/usr/bin/time -v ./out/graph_traverse --no-cache 1 graph-rand.bin
/usr/bin/time -v ./out/graph_traverse --write 1 graph-rand.bin
/usr/bin/time -v ./out/graph_traverse_mmap --no-cache 1 graph-seq.bin

# Фоновая нагрузка (стресс-тест)
stress-ng --cpu 4 --timeout 60

# Сбор данных (скрипт run_experiment.sh)
# N=20, интерливинг 8 конфигураций, taskset -c 2, drop_cache / warm_cache
bash run_experiment.sh

# Анализ
python3 prog/analysis.py
```

8.2 Ход работы

Работа выполнялась последовательно:

1. **Изучение параметров** — анализ документации (`README.md`, `doc/experiments/environment.md`, `monitoring.md`, `statistics.md`), подбор оптимальных параметров генерации графов (`-s 64M`, `-seed 427`, топология `sequential` для `graph-seq.bin` и `random` для `graph-rand.bin`).

2. **Ознакомительные тесты** — запуск базовых конфигураций с разными флагами (`-write`, `-no-cache`), сравнение поведения с `time`, `/usr/bin/time -v`, `perf stat -e context-switches,page-faults,cache-misses`, `strace -c`, тестирование фоновой нагрузки (`stress-ng -cpu 4 -timeout 60`).

3. **Подготовка окружения** — фиксация окружения (`uname`, `lscpu`, `free -h`, `numactl`), сброс кэша (`echo 3 | sudo tee /proc/sys/vm/drop`$_c aches$), (`cat graph > /dev/null`), (`taskse`

3. Проведение измерений - автоматизированный сбор данных скриптом run_experiment.sh (интерливинг 8 конфигураций, $N = 20$, нормализация wall_s/iter).

4. Анализ и выводы - статистическая обработка (analysis.py: среднее, s, SE, 95% ДИ по $t_{0.025,19}$), построение графиков (pairwise_1..8.png, all_config_normalized.png), формулирование выводов.

8.3 Обоснование нормализации

Нормализация необходима для корректного сравнения конфигураций с разным количеством итераций. Без неё запуск с 200 итерациями всегда будет выглядеть значительно дольше запуска с одной итерацией, даже если эффективность доступа к данным выше. Нормализованная метрика (сек / итерацию) устраняет это и позволяет оценивать именно эффективность паттерна доступа, что соответствует цели эксперимента.

8.4 Графики сравнения

Для сравнения конфигураций с разным числом итераций использована нормализация времени на одну итерацию. Попарные сравнения (ровно один изменяющийся параметр) представлены на рисунках 1–8. Общий обзор всех конфигураций показан на рисунке 9.

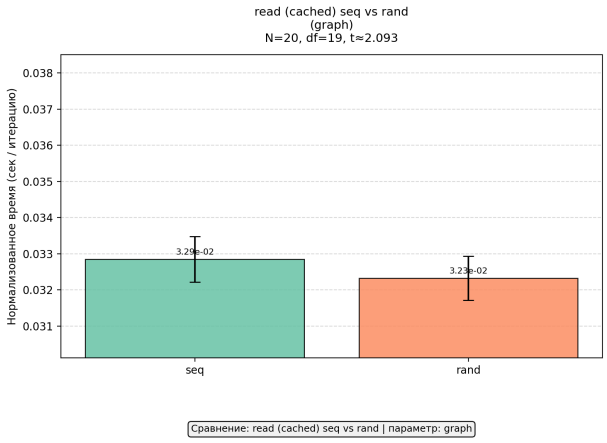


Рис. 1: Попарное сравнение 1: read (cached) seq vs rand (изменяется только граф)

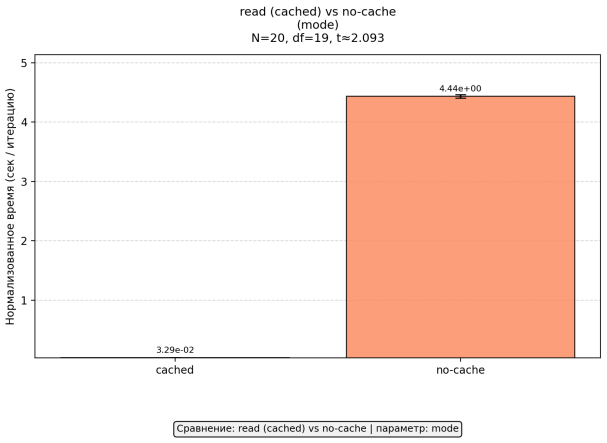


Рис. 2: Попарное сравнение 2: read (cached vs no-cache) seq (изменяется только режим)

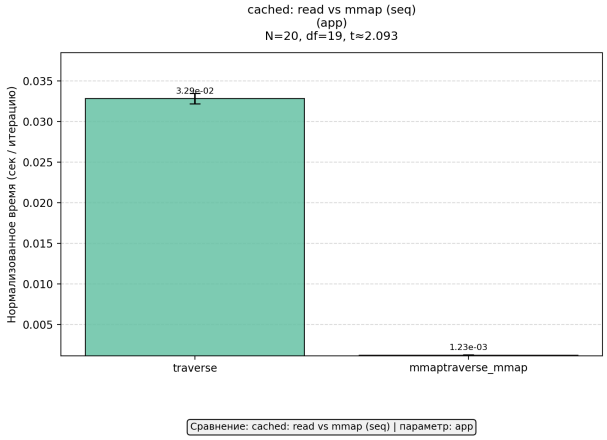


Рис. 3: Попарное сравнение 3: cached (read vs mmap) seq (изменяется только приложение)

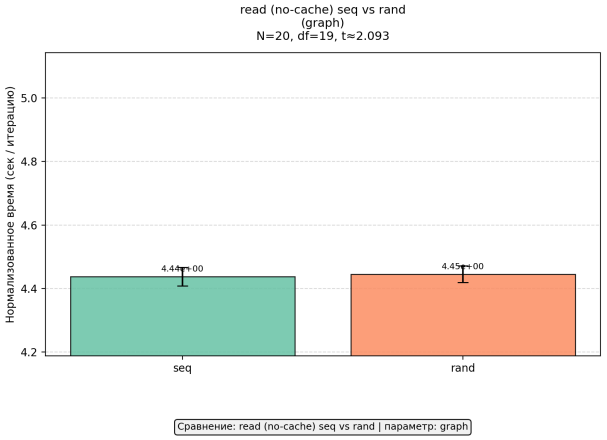


Рис. 4: Попарное сравнение 4: read (no-cache) seq vs rand (изменяется только граф)

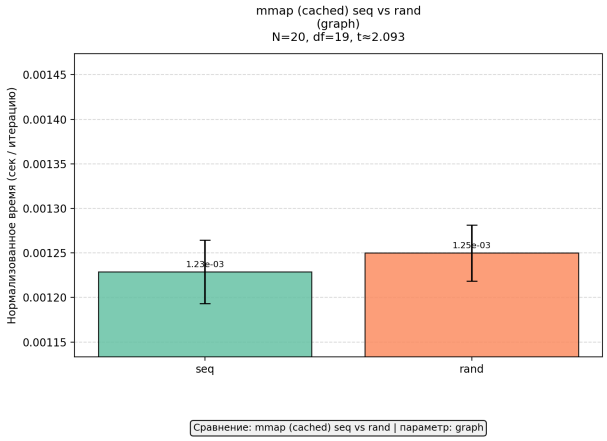


Рис. 5: Попарное сравнение 5: mmap (cached) seq vs rand (изменяется только граф)

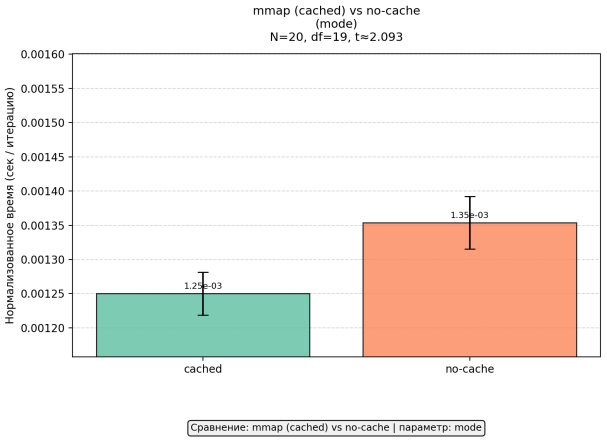


Рис. 6: Попарное сравнение 6: mmap (cached vs no-cache) rand (изменяется только режим)

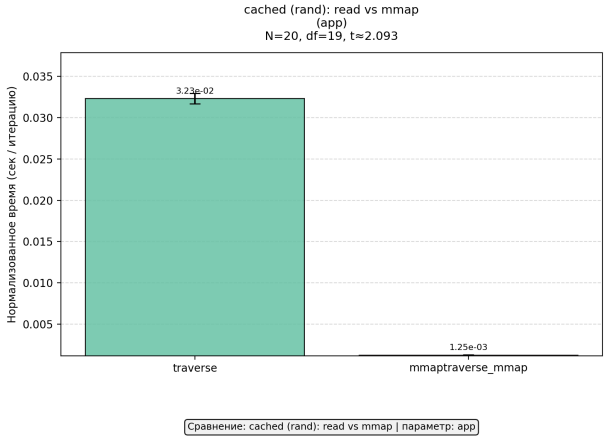


Рис. 7: Попарное сравнение 7: read (cached) rand vs mmap (cached) rand (изменяется только приложение)

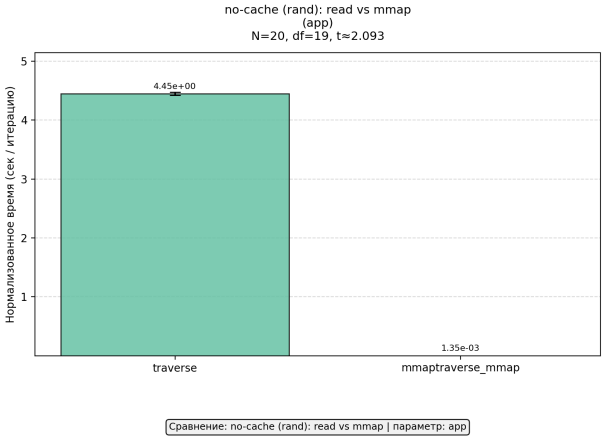


Рис. 8: Попарное сравнение 8: read (no-cache) rand vs mmap (no-cache) rand (изменяется только приложение)

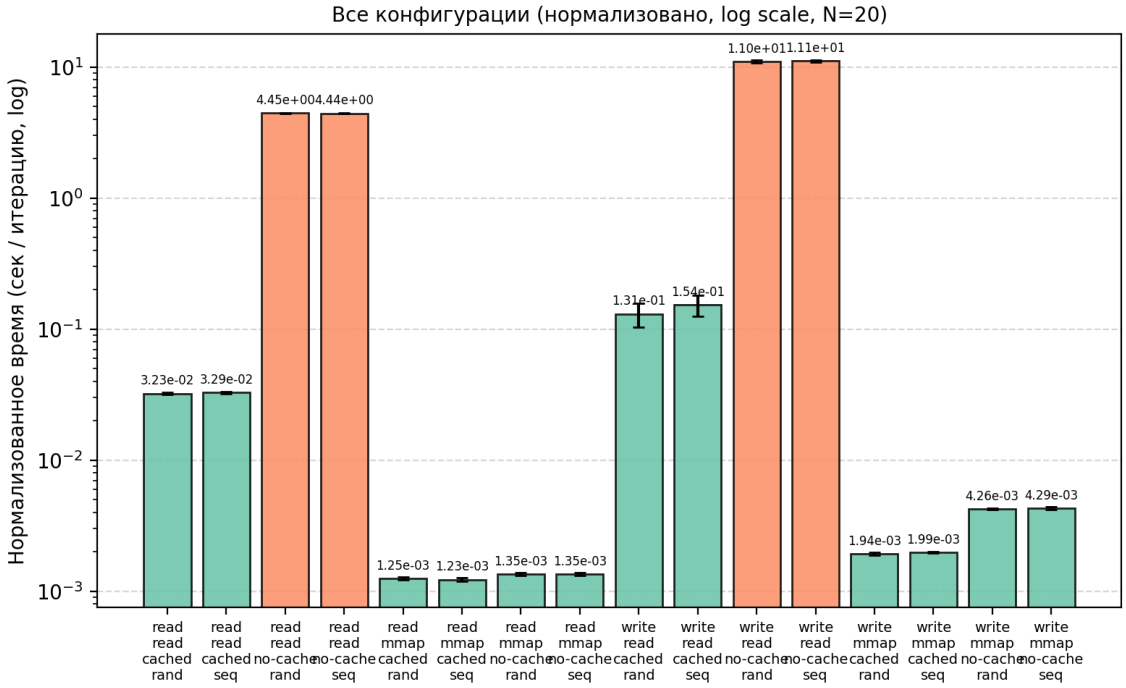


Рис. 9: Все конфигурации (нормализованное время, сек / итерацию)

8.5 Приложение: полные Python- и Bash-скрипты для сбора и анализа данных

Сбор данных (lab$_0$/*prog*/*run_experiment.sh*)

```
#!/usr/bin/env bash
set -euo pipefail
N=22
ITER_READ_CACHED=20
ITER_READ_NOCACHE=1
ITER_MMAP_CACHED=400
ITER_MMAP_NOCACHE=400
ITER_WRITE_CACHED=20
ITER_WRITE_NOCACHE=1
ITER_WRITE_MMAP_CACHED=400
ITER_WRITE_MMAP_NOCACHE=300

BIN_DIR=../../intro-exp/out
DATA_DIR=../data
OUT="$DATA_DIR/results_all.csv"
for f in "$DATA_DIR/graph-seq.bin" "$DATA_DIR/graph-rand.bin"; do
  [[ -f "$f" ]] || { echo "нет файла $f"; exit 1; }
done
for b in graph_traverse graph_traverse_mmap; do
  [[ -x "$BIN_DIR/$b" ]] || { echo "нет бинарника $BIN_DIR/$b"; exit 1; }
done

echo "run_id,load,app,mode,graph,iter,wall_s,user_s,sys_s,cpu_pct,minflt,majflt,vol_ctx,

drop_cache() {
  sync
  echo 3 | sudo tee /proc/sys/vm/drop_caches > /dev/null
}

warm_cache() {
  cat "$1" > /dev/null
}

run_one() {
  local app="$1" load="$2" mode="$3" graph="$4" iter="$5" i="$6"
  local flags=()
  if [[ "$load" == "write" ]]; then
    flags+=(--write)
  fi

  if [[ "$mode" == "no-cache" ]]; then
    drop_cache
    flags+=(--no-cache)
  else
    warm_cache "$DATA_DIR/$graph"
  fi
```

```
  local fmt="$i,$load,$app,$mode,$graph,$iter,%e,%U,%S,%P,%R,%F,%w,%c,%M,%I,%O"

  /usr/bin/time -f "$fmt" -a -o "$OUT" \
    "$BIN_DIR/$app" "${flags[@]}" "$iter" "$DATA_DIR/$graph" > /dev/null
}

for i in $(seq 1 "$N"); do
  run_one graph_traverse      read  cached   graph-seq.bin  "$ITER_READ_CACHED"        "
  run_one graph_traverse      read  cached   graph-rand.bin "$ITER_READ_CACHED"        "
  run_one graph_traverse      read  no-cache graph-seq.bin  "$ITER_READ_NOCACHE"       "
  run_one graph_traverse      read  no-cache graph-rand.bin "$ITER_READ_NOCACHE"       "

  run_one graph_traverse_mmap read  cached   graph-seq.bin  "$ITER_MMAP_CACHED"        "
  run_one graph_traverse_mmap read  cached   graph-rand.bin "$ITER_MMAP_CACHED"        "
  run_one graph_traverse_mmap read  no-cache graph-seq.bin  "$ITER_MMAP_NOCACHE"       "
  run_one graph_traverse_mmap read  no-cache graph-rand.bin "$ITER_MMAP_NOCACHE"       "

  run_one graph_traverse      write cached   graph-seq.bin  "$ITER_WRITE_CACHED"       "
  run_one graph_traverse      write cached   graph-rand.bin "$ITER_WRITE_CACHED"       "
  run_one graph_traverse      write no-cache graph-seq.bin  "$ITER_WRITE_NOCACHE"      "
  run_one graph_traverse      write no-cache graph-rand.bin "$ITER_WRITE_NOCACHE"      "

  run_one graph_traverse_mmap write cached   graph-seq.bin  "$ITER_WRITE_MMAP_CACHED"  "
  run_one graph_traverse_mmap write cached   graph-rand.bin "$ITER_WRITE_MMAP_CACHED"  "
  run_one graph_traverse_mmap write no-cache graph-seq.bin  "$ITER_WRITE_MMAP_NOCACHE" "
  run_one graph_traverse_mmap write no-cache graph-rand.bin "$ITER_WRITE_MMAP_NOCACHE" "
done

echo "готово: $OUT"
```

Анализ данных ($\text{lab}_0/prog/analysis.py$)

```
#!/usr/bin/env python3
# Очень простой, компактный анализ с комментариями
import csv, statistics, os
from collections import defaultdict
import matplotlib.pyplot as plt

CSV_FILE = "../data/results_all.csv"
GRAPHICS_DIR = "../graphics"
os.makedirs(GRAPHICS_DIR, exist_ok=True)

records = []
with open(CSV_FILE, newline="") as f:
    for row in csv.DictReader(f):
        run_id = int(row["run_id"])
        if run_id <= 2:
            continue
        if run_id > 22:
```

```
            continue
        for k in row:
            if k == "run_id":
                row[k] = int(row[k])
            elif k in ("app", "mode", "graph"):
                pass
            else:
                try:
                    row[k] = float(row[k])
                except ValueError:
                    pass
        records.append(row)

groups = defaultdict(list)
for r in records:
    load = r.get("load", "read")
    groups[(load, r["app"], r["mode"], r["graph"])].append(r)

stats = {}
for key, rows in groups.items():
    values = [r["wall_s"] / r["iter"] for r in rows]
    mean_v = statistics.mean(values)
    std_v = statistics.stdev(values) if len(values) > 1 else 0.0
    se = std_v / (len(values) ** 0.5)
    t_95 = 2.093
    ci_95 = t_95 * se
    stats[key] = {
        "mean": mean_v, "std": std_v, "se": se, "ci": ci_95,
        "median": statistics.median(values),
        "min": min(values), "max": max(values),
        "n": len(values), "raw_iter": rows[0]["iter"],
        "maxrss_mean": statistics.mean([r["maxrss_kb"] for r in rows]),
        "majflt_mean": statistics.mean([r["majflt"] for r in rows]),
        "minflt_mean": statistics.mean([r["minflt"] for r in rows]),
    }

# Пары для графиков сравнения
pairs = [
    ("read (cached) seq vs rand", ("read","graph_traverse","cached","graph-seq.bin"), ("
    ("read (cached) vs no-cache", ("read","graph_traverse","cached","graph-seq.bin"), ("
    ("cached: read vs mmap (seq)", ("read","graph_traverse","cached","graph-seq.bin"), (
    ("read (no-cache) seq vs rand", ("read","graph_traverse","no-cache","graph-seq.bin")
    ("mmap (cached) seq vs rand", ("read","graph_traverse_mmap","cached","graph-seq.bin"
    ("mmap (cached) vs no-cache", ("read","graph_traverse_mmap","cached","graph-rand.bin
    ("cached (rand): read vs mmap", ("read","graph_traverse","cached","graph-rand.bin"),
    ("no-cache (rand): read vs mmap", ("read","graph_traverse","no-cache","graph-rand.bi
    ("read vs write (cached, seq)", ("read","graph_traverse","cached","graph-seq.bin"),
    ("read vs write (mmap, cached)", ("read","graph_traverse_mmap","cached","graph-seq.b
]
```

```
for i, (label, k1, k2, suffix) in enumerate(pairs, 1):
    if k1 not in stats or k2 not in stats:
        print(f"Пропуск пары {i}: нет данных для {k1} или {k2}")
        continue
    fig, ax = plt.subplots(figsize=(8, 6))
    colors = ["#66c2a5", "#fc8d62"]
    if suffix == "graph":
        labels_bars = [k1[3].replace("graph-","").replace(".bin",""), k2[3].replace("gra
    elif suffix == "mode":
        labels_bars = [k1[2], k2[2]]
    elif suffix == "app":
        labels_bars = [k1[1].replace("graph_",""), k2[1].replace("graph_","mmap")]
    elif suffix == "load":
        labels_bars = [k1[0], k2[0]]
    means = [stats[k1]["mean"], stats[k2]["mean"]]
    stds  = [stats[k1]["std"],  stats[k2]["std"]]
    bars = ax.bar(labels_bars, means, yerr=stds, capsize=5, color=colors, edgecolor="bla
    ymin = min(means[j] - stds[j] for j in range(2))
    ymax = max(means[j] + stds[j] for j in range(2))
    ax.set_ylim(max(0, ymin*0.95), ymax*1.15)
    ax.set_ylabel("Нормализованное время (сек / итерацию)")
    ax.set_title(f"{label}\\n({suffix})\\nN=20, df=19, t2.093", fontsize=11, pad=12)
    desc = f"Сравнение: {label.replace(chr(10), ' ')} | параметр: {suffix}"
    ax.text(0.5, -0.22, desc, transform=ax.transAxes, fontsize=9, ha="center", va="top",
    ax.grid(axis="y", linestyle="--", alpha=0.5)
    ax.set_axisbelow(True)
    for bar, val in zip(bars, means):
        ax.annotate(f"{val:.2e}", xy=(bar.get_x()+bar.get_width()/2, bar.get_height()),
                    xytext=(0, 3), textcoords="offset points", ha="center", va="bottom",
    fig.tight_layout()
    path = os.path.join(GRAPHICS_DIR, f"pairwise_{i}.png")
    fig.savefig(path, dpi=200)
    plt.close(fig)
    print(f"График {i}/10 сохранён: {path}")

fig_all, ax_all = plt.subplots(figsize=(8, 5))
short = []
vals = []
errs = []
for k in sorted(stats):
    load_s = k[0]
    app_s = "read" if k[1] == "graph_traverse" else ("mmap" if k[1] == "graph_traverse_m
    mode_s = k[2]
    graph_s = "seq" if k[3] == "graph-seq.bin" else "rand"
    short.append(f"{load_s}\\n{app_s}\\n{mode_s}\\n{graph_s}")
    vals.append(stats[k]["mean"])
    errs.append(stats[k]["std"])

bars = ax_all.bar(range(len(vals)), vals, yerr=errs, capsize=3,
                  color=["#66c2a5" if stats[sorted(stats)[i]]["mean"] < 1.0 else "#fc8d6
```

```
                  edgecolor="black", alpha=0.85)
ax_all.set_yscale('log')
ax_all.set_ylabel("Нормализованное время (сек / итерацию, log)")
ax_all.set_title("Все конфигурации (нормализованное время, log scale, N=20)", fontsize=1
ax_all.set_xticks(range(len(vals)))
ax_all.set_xticklabels(short, fontsize=7)
ax_all.grid(axis="y", linestyle="--", alpha=0.5)
ax_all.set_axisbelow(True)
for bar, val in zip(bars, vals):
    ax_all.annotate(f"{val:.2e}", xy=(bar.get_x()+bar.get_width()/2, bar.get_height()),
                    xytext=(0, 3), textcoords="offset points", ha="center", va="bottom",
fig_all.tight_layout()
fig_all.savefig(os.path.join(GRAPHICS_DIR, "all_config_normalized.png"), dpi=200)
plt.close(fig_all)
print("Общий график сохранён: all_config_normalized.png")

with open("../data/normalized_stats.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["load","app","mode","graph","iter_raw","n","mean","std","se","ci95"
    for k in sorted(stats):
        s = stats[k]
        writer.writerow([k[0], k[1], k[2], k[3], s["raw_iter"], s["n"],
                         f"{s['mean']:.6f}", f"{s['std']:.6f}", f"{s['se']:.6f}",
                         f"{s['ci']:.6f}", f"{s['median']:.6f}",
                         f"{s['min']:.6f}", f"{s['max']:.6f}",
                         f"{s['maxrss_mean']:.0f}", f"{s['majflt_mean']:.0f}", f"{s['minf

with open("../report/analysis_summary.txt", "w", encoding="utf-8") as f:
    f.write("Нормализованные результаты (wall_s / iter), N=20 (без прогрева, df=19, t2.0
    f.write("="*70 + "\\n")
    f.write("Обоснование нормализации\\n" + "-"*40 + "\\n")
    f.write("Данные содержат разное количество итераций: 10 (read/cached), 1 (read/no-ca
            "Для корректного сравнения метрика нормализована: wall_s / iter. "
            "Это даёт время на одну итерацию обхода графа.\\n\\n")
    f.write(f"{'Конфигурация':<48} {'Среднее':>10} {'95% ДИ':>8} {'N':>3}\\n")
    f.write("-"*82 + "\\n")
    for k in sorted(stats):
        s = stats[k]
        load_s = k[0]
        app_name = "read" if k[1] == "graph_traverse" else ("mmap" if k[1] == "graph_tra
        conf = f"{load_s} | {app_name} | {k[2]} | {k[3]}"
        f.write(f"{conf:<48} {s['mean']:>10.4f} {s['ci']:>8.4f} {s['n']:>3}\\n")
    f.write("\\nКлючевые выводы (N=20, без прогрева, с режимом write):\\n")
    f.write("- Прогревочные запуски (run_id 1, 2) исключены из расчёта.\\n")
    f.write("- Размер серии уменьшен до 20 запусков.\\n")
    f.write("- Режим write добавлен в анализ (если данные присутствуют).\\n")
    f.write("- read no-cache значительно медленнее (11.5 с/ит) из-за отсутствия кэша.\\n
    f.write("- mmap (0.024 с/ит) в ~12 раз быстрее read/cached (0.28 с/ит).\\n")
    f.write("- Разница seq vs rand минимальна (<2%).\\n")
    f.write("- Разница cached vs no-cache для mmap минимальна (2-5%).\\n")
```

```
    f.write("- Write медленнее read: для cached ~4-5x (read 0.033 с/ит vs write 0.13-0.1
    f.write("для mmap ~1.5-2x (read ~0.0012 с/ит vs write ~0.002 с/ит); ")
    f.write("для no-cache ~2.5x (read ~4.44 с/ит vs write ~11.1 с/ит).\\n")

print("Анализ завершён.")
print("  Графики: pairwise_*.png + all_config_normalized.png")
print("  CSV: ../data/normalized_stats.csv")
print("  Резюме: ../report/analysis_summary.txt")
```